



**Our first program**

## Our goal

Add the first 10 integers together and calculate the sum in C, C++, and Rust

# Caveat

We will concentrate on the basic translation from text to binary

Not the particulars of the code yet

# Heart of the Computer

A computer's main function is to process data

# Heart of the Computer

The computer performs data processing in the data path

# Heart of the Computer

Data operations are performed on data  
in the **datapath**

In our case, the operation is add

# Generalize Operation Processing

All types of operation are possible on data in the **datapath**

We can do arithmetic like **add**

We can do logic like **and**

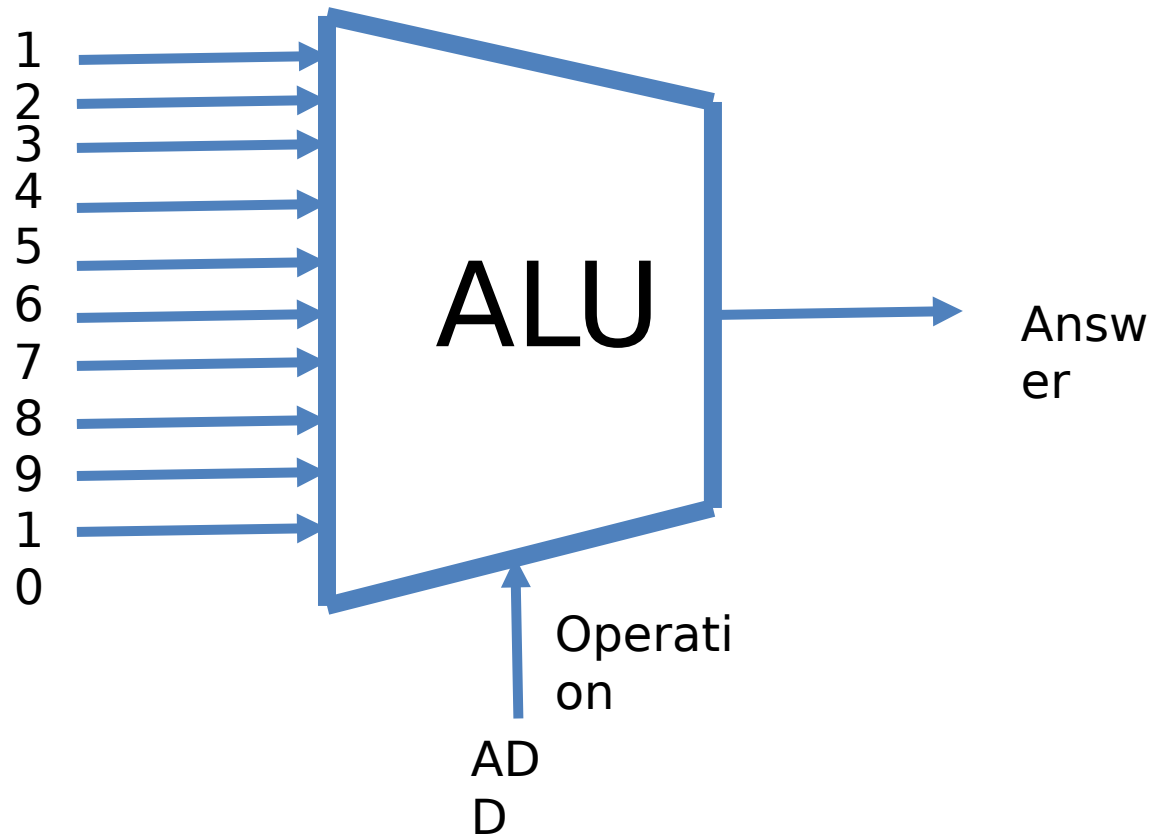
# Generalize Operation Processing

We can make a generalize circuit that does arithmetic and logic

This is called the Arithmetic Logic Unit or ALU

It is the heart of the computer

# ALU

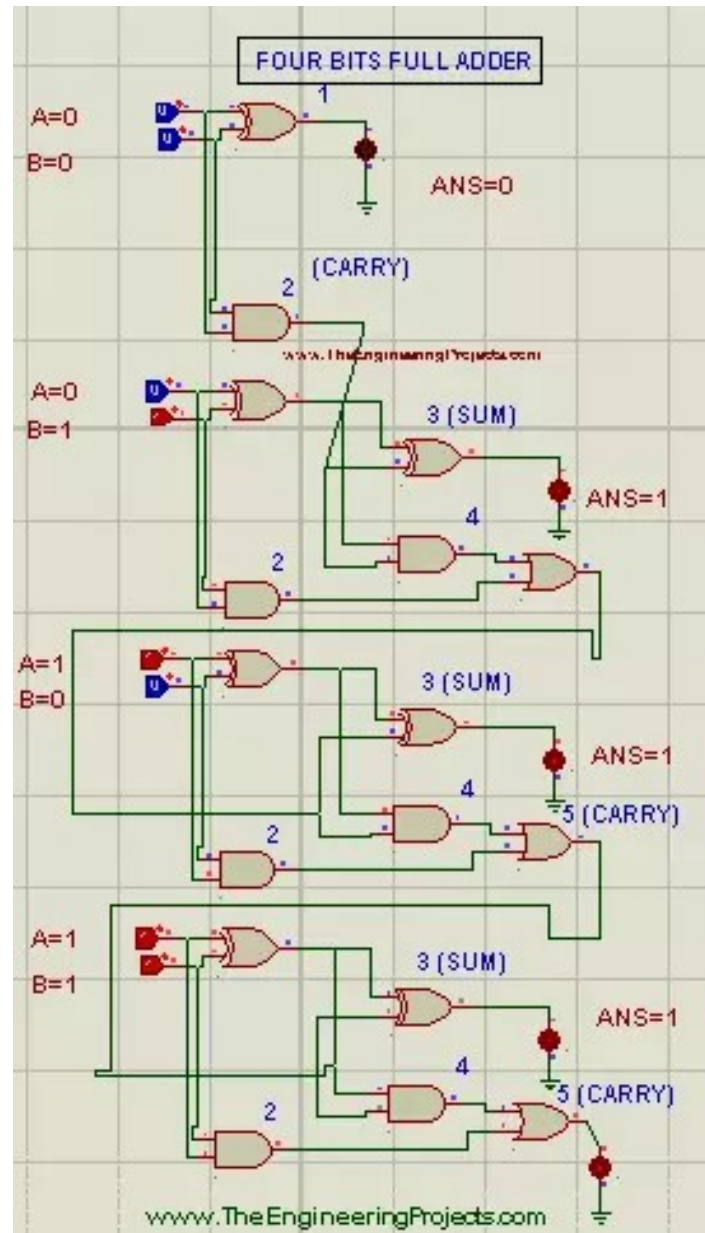


# Building an ALU

We can design an ALU using digital electronic circuits known as gates

Today we can synthesize ALUs using computer aided design tools known as CAD tools

# 4 Bit Adder Circuit



# Building a Computer

Computer can be thought as completing a sequence of steps

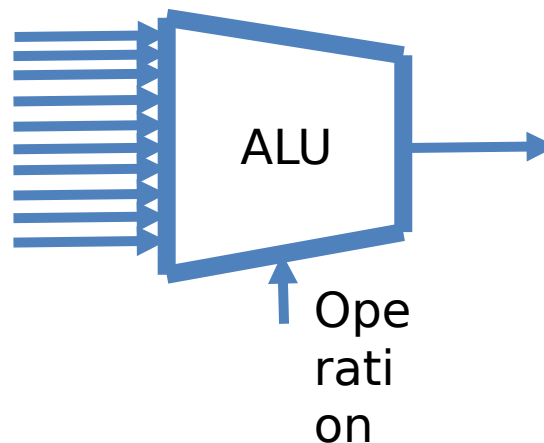
Get an instruction (Fetch)

Figure out what to do (Decode)

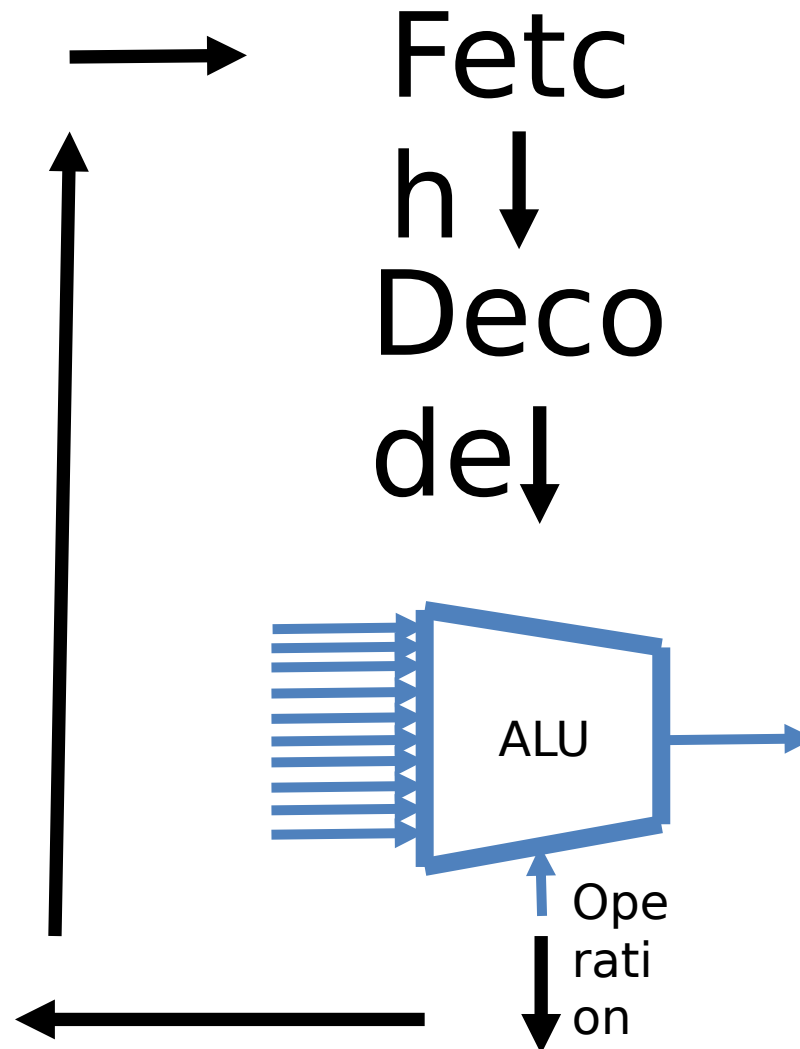
Perform the instruction  
(Execute)

# Executing an Instruction

Fetch  
↓  
Decode

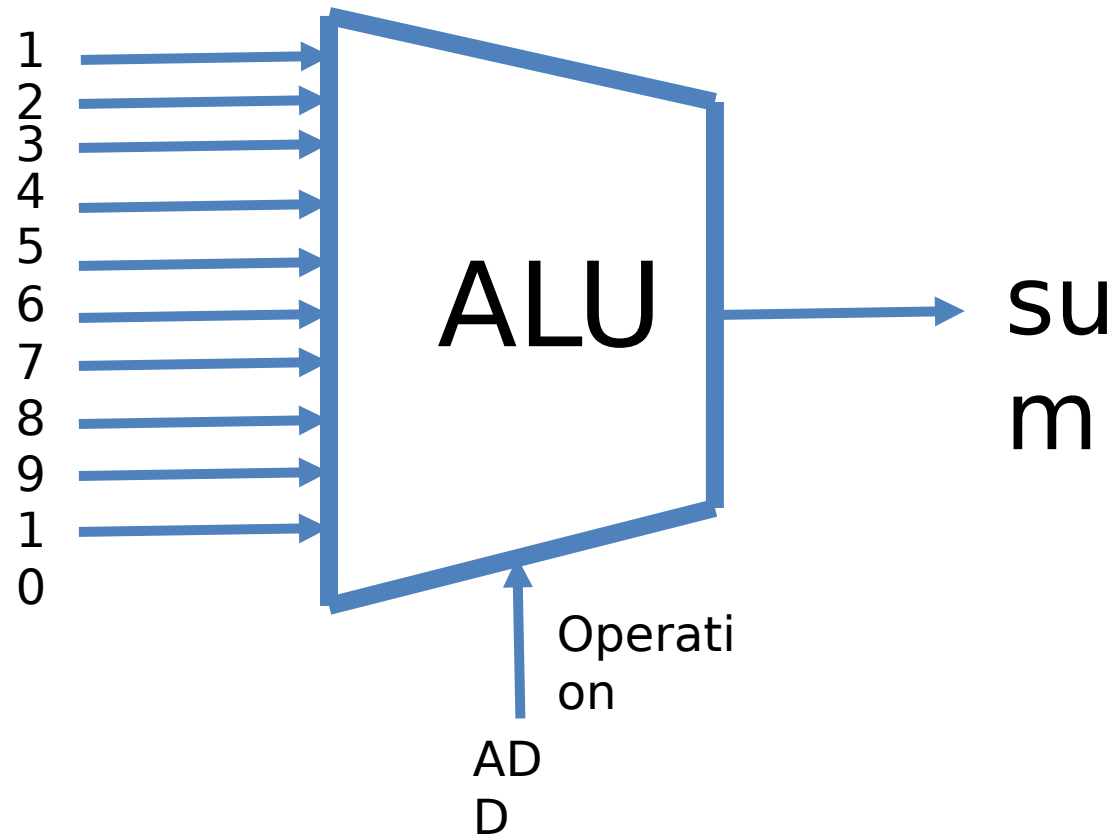


# Do it again!



# Memory

We need to remember a calculation or state

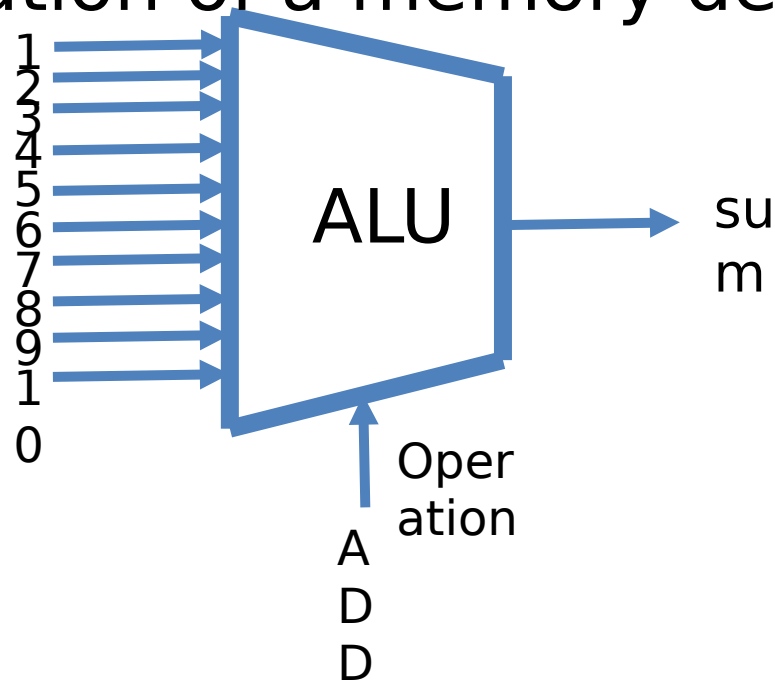


# Memory

Physically have to put the results of the add somewhere

That place is called the address of sum

It is the location of a memory device



# Location of Storage

Address is the location of memory or symbol (variable)

The computer knows them as unsigned integers

What are integers and unsigned integers?

Let's review

# Location of Storage

Address is the location of memory or symbol (variable)

The computer knows them as unsigned integers

# Location of Storage

Address is the location of memory or symbol (variable)

The computer knows them as unsigned integers

Unsigned integers start at 0 and go to  $2^{\text{numbits}} - 1$  in binary

Too difficult for humans to keep track of these numbers

~~Assembly language~~



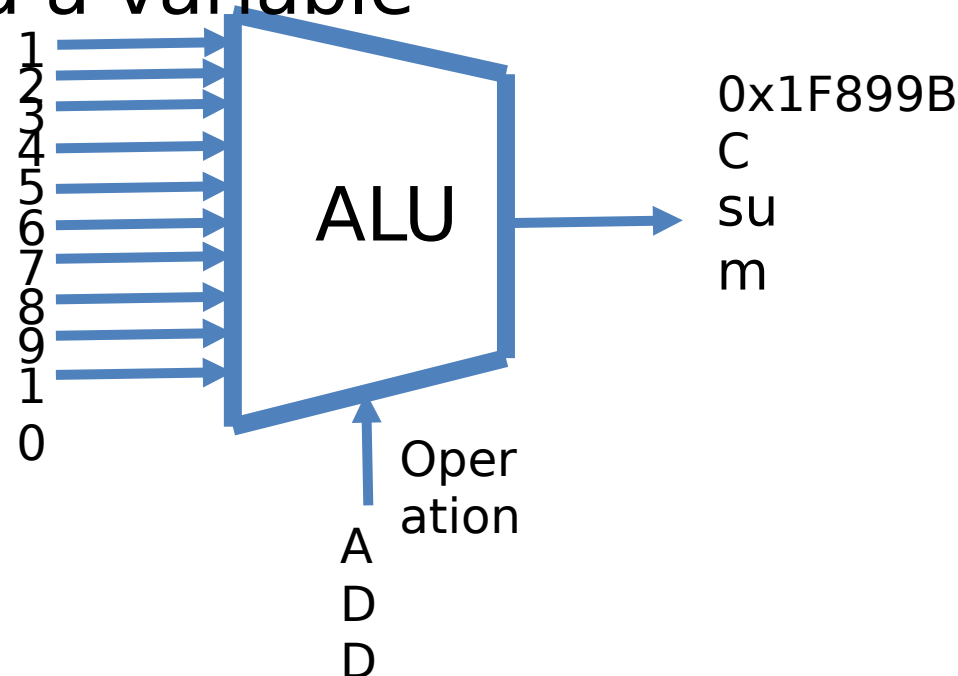
0x1F899BC

# Symbol

To make it easier, we associate a symbolic name with the address

This is known as a symbol

Loosely called a variable



# But not the value

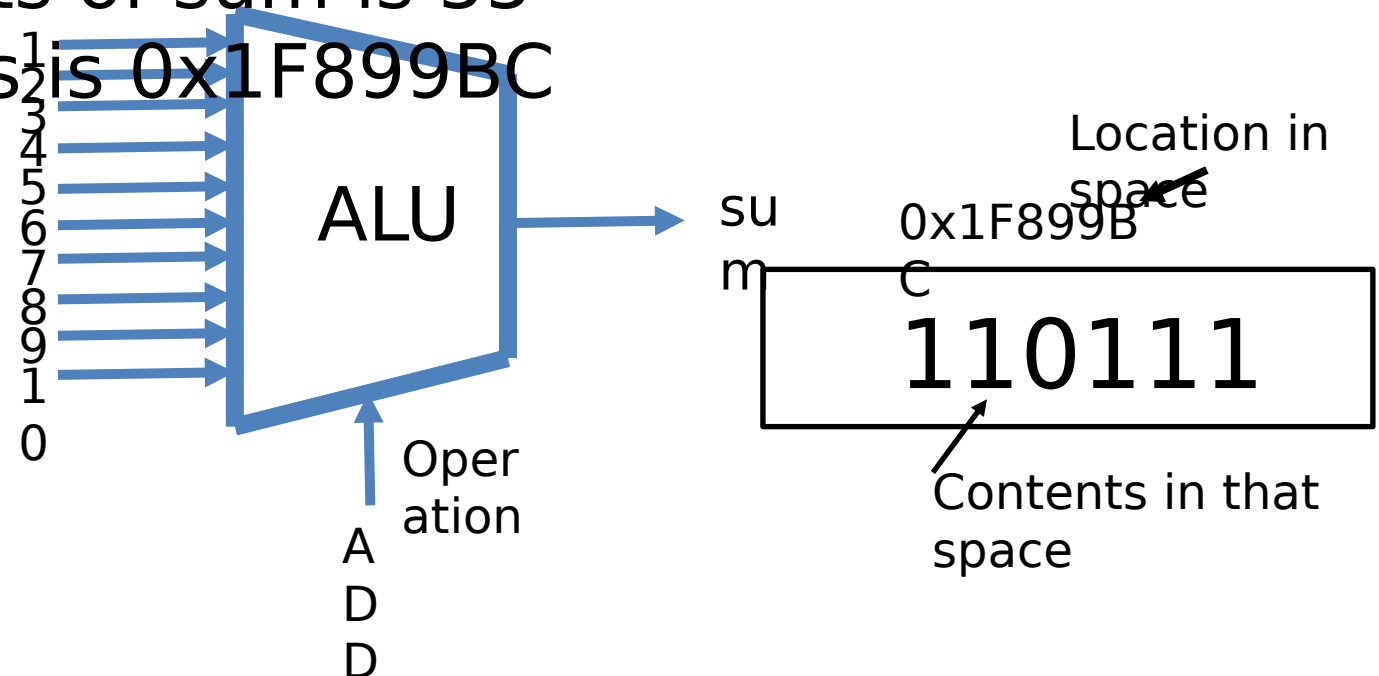
The symbol is not the  
computed value

The address is a location

We need the contents of the  
address

The contents of sum is 55

The address is 0x1F899BC



# Making an association

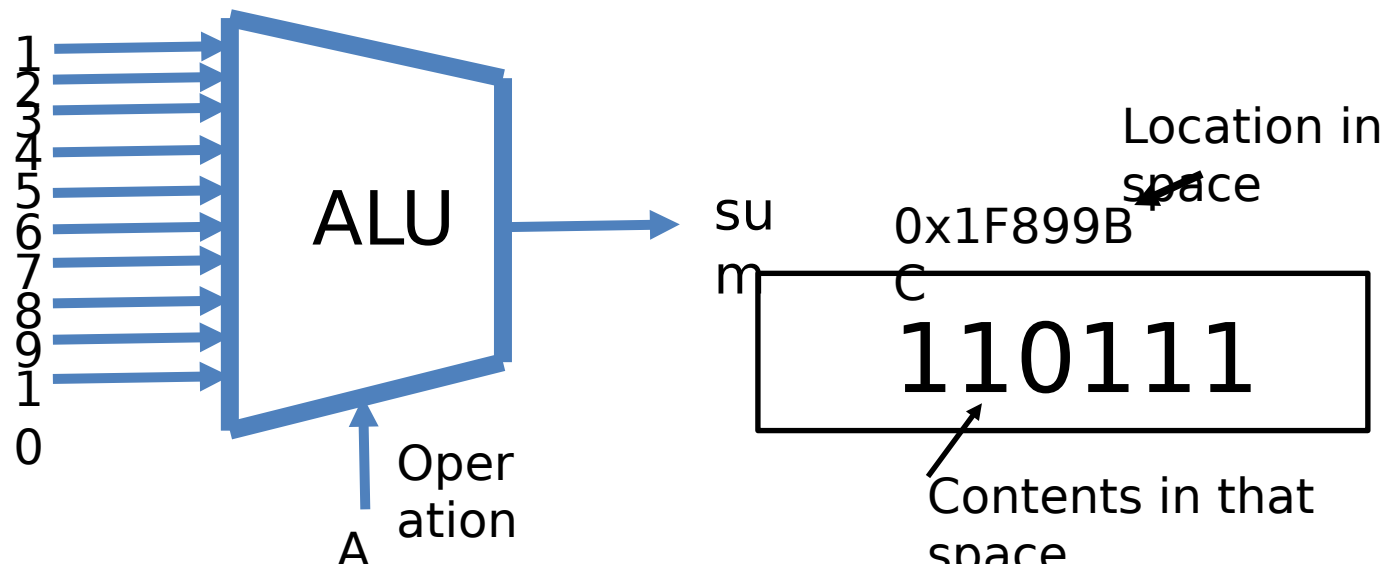
Symbolically make an association.

Let

sum be the address or location of memory

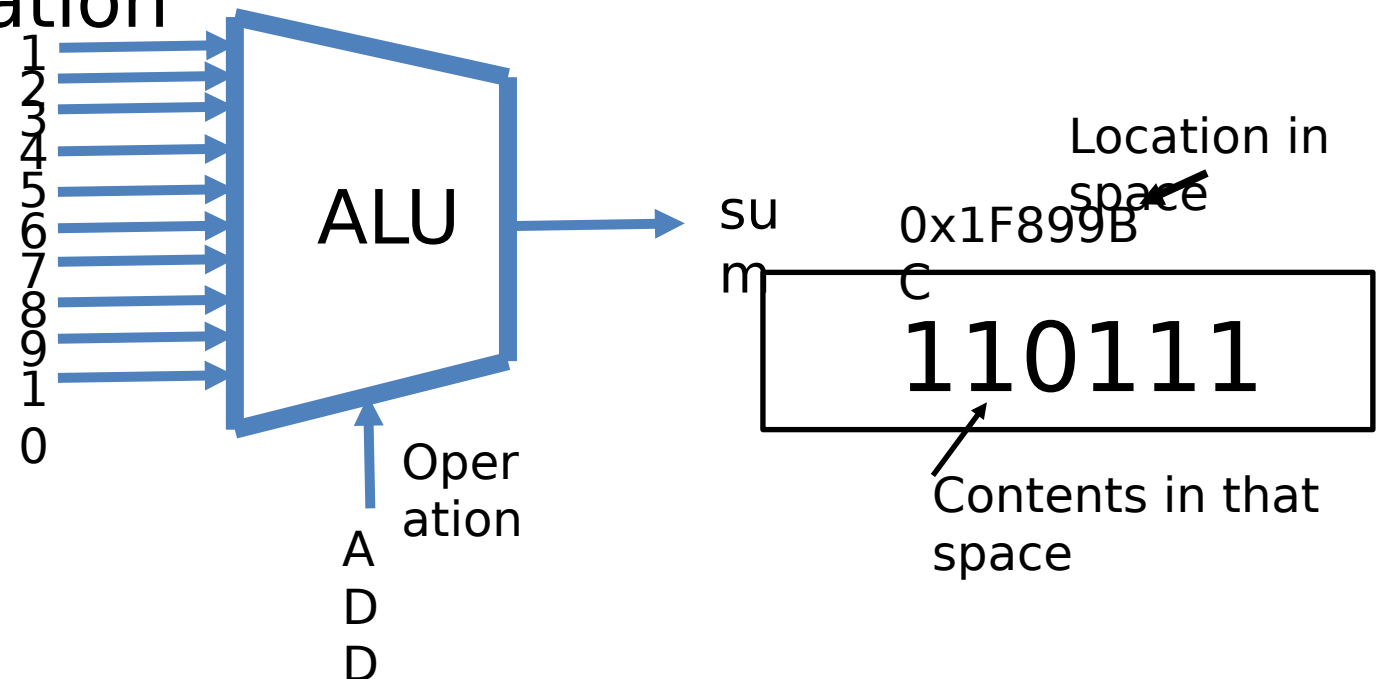
\$sum be the contents of memory

Very clean and exact



# Humans hate to type

Most of the time we want to process data  
We really don't want to care about the location  
We just want the answer  
So just drop the address from consideration  
The symbol represents the contents of a  
memory location



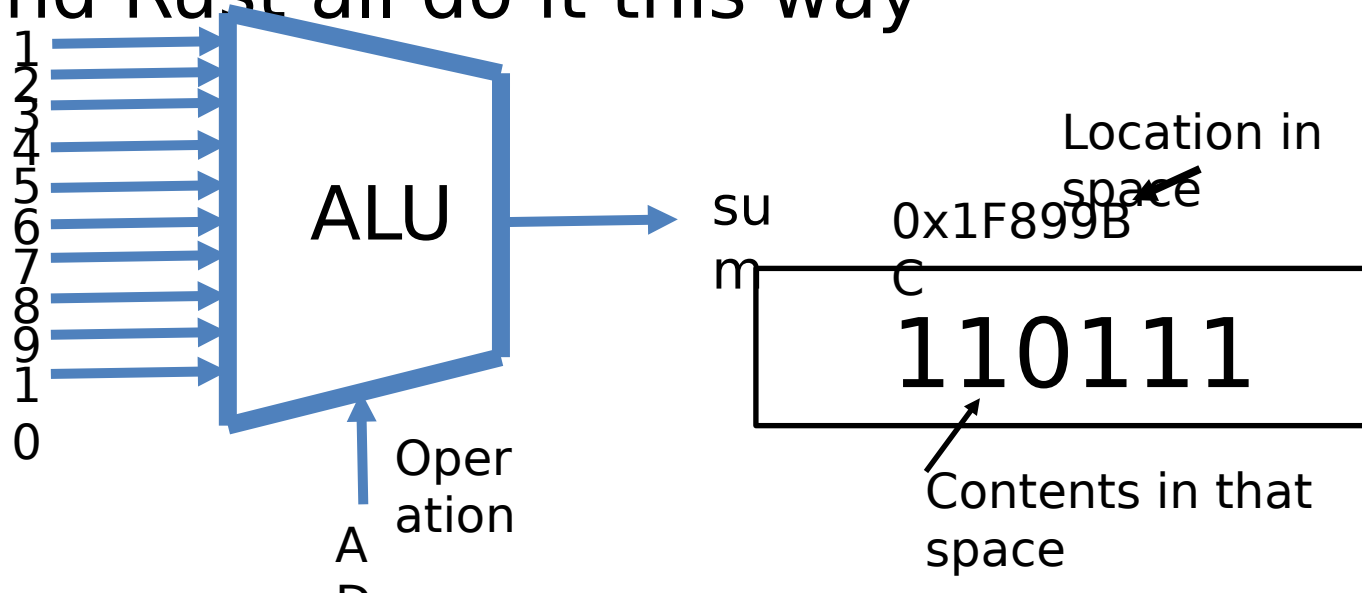
# Humans hate to type

If we want the address, we will use the & address operator

&sum is 0x1F899BC

sum is 11011 in binary or 55

C, C++, and Rust all do it this way



# Our goal

Now that we see what is going on inside the computer, let's see how to program it.

Add the first 10 integers together and calculate the sum in C, C+, and Rust

# Our Program Location

We will write human readable text for our programs using an **editor**

It will be stored in a **file** which is a collection of contiguous bytes

# Editors

Many programs to modify code

VSCode, vi, emacs, gedit...

# Building Our Program

We will now show this process of how text in a file is converted to machine code

# Compiler/Linker

All three compiled languages can compile and link in a single step for simple programs

Under the hood, it is multiple steps  
compiling  
linking

# Compiling

Translate human text statements to binary code

All three have a similar form on the command line

`<compiler_name> <options> <file_to_compile>`

# Compiling First

Let's call the program “first”

# Compiling First in C

In C, our human text will reside in a file called “first.c”

The file has a suffix of .c

# Compiling First in C++

In C++, our human text will reside in a file called “first.cpp”

Sometimes the suffix for C++ is cxx.

# Compiling First in Rust

In Rust+, our human text will reside in a file called “first.rs”

# Compiling

Translate human text statements to binary code

All three have a similar form on the command line

`<compiler_name> <options> <file_to_compile>`

# Compiling C on Command Line

`<compiler_name> <options> <file_to_compile>`

`gcc -o first first.c`

Options: `-o first` : output program should be named first

# Compiling C++ on Command Line

`<compiler_name> <options> <file_to_compile>`

`g++ -o first first.cpp`

Options: `-o first` : output program should be named first

# Compiling Rust on Command Line

`<compiler_name> <options> <file_to_compile>`

`rustc -o first first.rs`

Options: `-o first` : output program should be named first

# First Text in C

```
int main( int argc, char *argv[] )  
{  
    int sum = 1 + 2 + 3 + 4 + 5 + 6 + 7  
    +8 + 9 + 10 ;  
}
```

# First Text in C

```
Entry  
point  
int main( int argc, char *argv[] )  
{  
    int sum = 1 + 2 + 3 + 4 + 5 + 6 + 7  
    +8 + 9 + 10 ;  
}
```

# First Text in C

Parameters or arguments associated with

entry point

```
int main( int argc, char *argv[] )  
{  
    int sum = 1 + 2 + 3 + 4 + 5 + 6 + 7  
    + 8 + 9 + 10 ;  
}
```

# First Text in C

Return value is a  
integer

```
int main( int argc, char *argv[] )  
{  
    int sum = 1 + 2 + 3 + 4 + 5 + 6 + 7  
    + 8 + 9 + 10 ;  
}
```

# First Text in C

```
int main( int argc, char *argv[] )  
{  
    int sum = 1 + 2 + 3 + 4 + 5 + 6 + 7  
    +8 + 9 + 10 ;  
}
```

sum is a variable that is a  
integer

# First Text in C

```
int main( int argc, char *argv[] )  
{  
    int sum = 1 + 2 + 3 + 4 + 5 + 6 + 7  
    +8 + 9 + 10 ;  
}
```

Statement  
separator

# First Text in C++

```
int main() {  
    int sum = 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 ;  
}
```

# First Text in Rust

```
fn main() {  
    let sum = 1 + 2 + 3 + 4 + 5 + 6 + 7  
+ 8 + 9 + 10 ;  
}
```

# Let's build the programs

Try each language in turn

# Programs don't do anything

We don't see the answer

We need Input and Output (I/O)

Let's add I/O

# Adding I/O in C

```
fprintf( stdout, "The sum is %d\n", sum  
) ;
```

# Adding I/O in C

That doesn't work

Need to define the fprintf function call.

We use a predefined file which is included using the preprocessor

```
#include <stdio.h>
```

<> Means to search for file in system directories such as /usr/include

# Adding I/O in C++

```
std::cout << "The sum of the first 10  
integers is: " << sum << std::endl;
```

# Adding I/O in C++

That doesn't work

Again we need to define **std**

```
#include <iostream>
```

Works like C, just a different set of objects

# Adding I/O in Rust

```
println!("The total sum is: {}", sum);
```

Hey, that works!

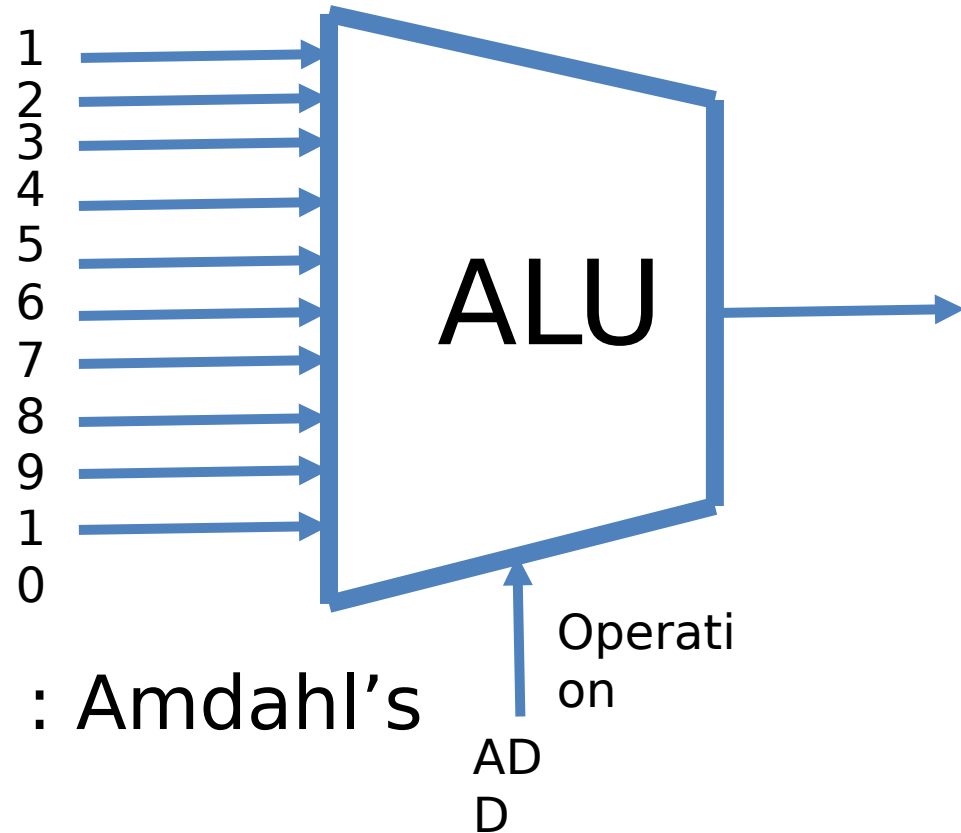
Println is an integral part of the Rust language

In C and C++, I/O was an external addition

Every machine could have a different way

# 10 input ALU

How often are we going to add exactly 10 numbers???

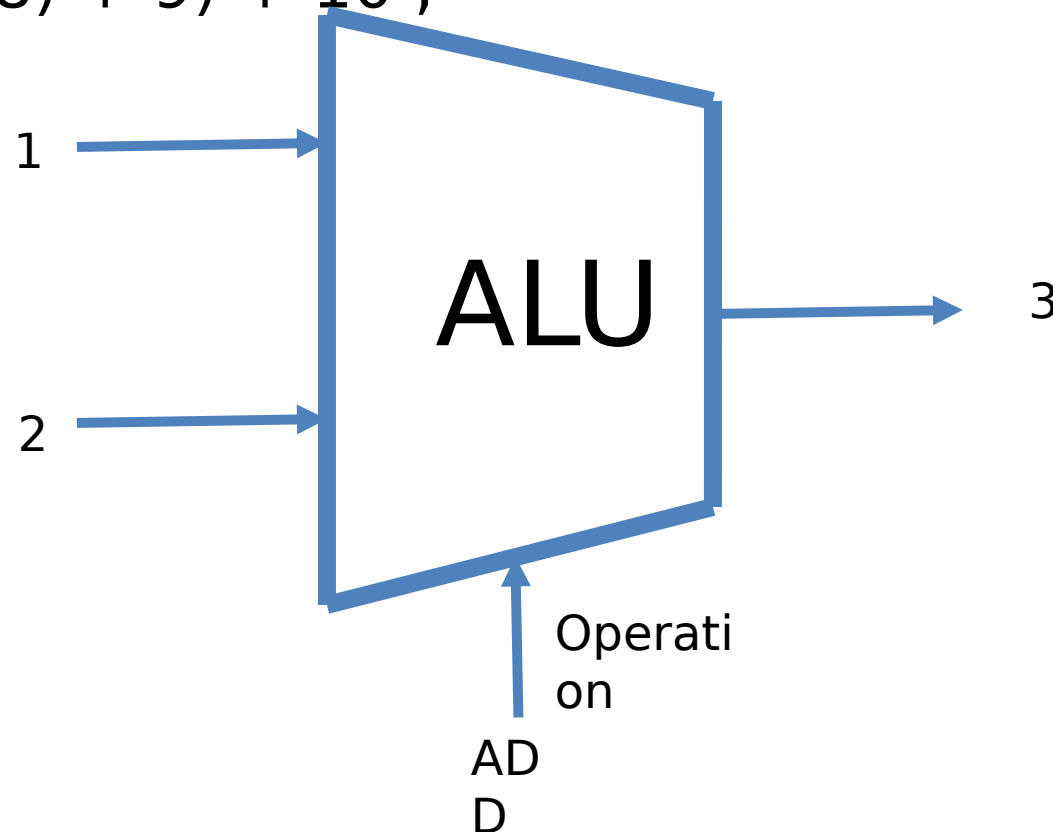


Not often : Amdahl's Law

Design for the

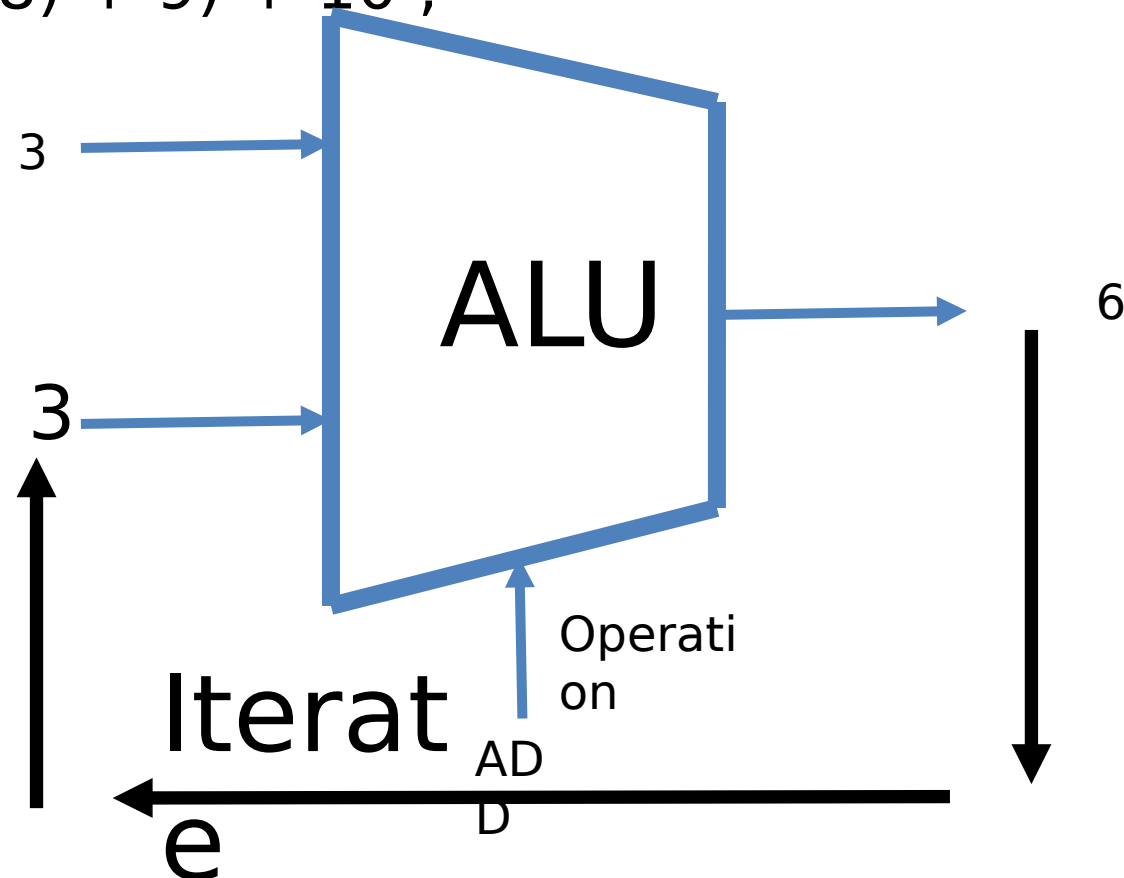
# Efficient ALU

sum = (((((((1 + 2) + 3) + 4) + 5) + 6) + 7) + 8) + 9) + 10 ;



# Accumulate

sum = (((((((1 + 2) + 3) + 4) + 5) + 6) + 7) + 8) + 9) + 10 ;



# Compiling Loop

Let's call the program "loop"

Let's use a variable to help iterate or accumulate a result

# Compiling Loop in C

In C, our human text will reside in a file called “loop.c”

# Compiling Loop in C++

In C++, our human text will reside in a file called “loop.cpp”

# Compiling Loop in Rust

In Rust+, our human text will reside in a file called “loop.rs”

# Compiling

Translate human text statements to binary code

All three have a similar form on the command line

`<compiler_name> <options> <file_to_compile>`

# Compiling C on Command Line

<compiler\_name> <options> <file\_to\_compile>

```
gcc -o loop loop.c
```

Options: -o loop : output program should be named loop

# Compiling C++ on Command Line

`<compiler_name> <options> <file_to_compile>`

`g++ -o loop loop.cpp`

Options: `-o loop` : output program should be named loop

# Compiling Rust on Command Line

<compiler\_name> <options> <file\_to\_compile>

```
rustc -o loop loop.rs
```

Options: -o loop : output program should be named loop

# Loop Text in C

```
#include <stdio.h>
```

```
int main( int argc, char *argv[] )
```

```
{
```

```
    int sum = 0 ;
```

```
    for( int i = 1 ; i <= 10 ; i++ ){
```

```
        sum += i ;
```

```
    }
```

```
    fprintf( stdout, "The sum is %d\n",  
sum ) ;
```

```
}
```

# Loop Text in C++

```
#include <iostream>
```

```
int main() {  
    int sum = 0;
```

```
    for (int i = 1; i <= 10; i++ ) {  
        sum += i;  
    }
```

```
    std::cout << "The sum of the first 10 integers is: " <<  
sum << std::endl;  
    return 0;
```

# Loop Text in Rust

```
fn main() {  
    let lo = 1 ;  
    let hi = 10 ;  
    let mut sum = 0 ;  
    for i in lo..=hi {  
        sum += i ;  
    }  
    println!("The total sum is: {}",  
sum);  
}
```

# Let's build the programs

Try each language in turn

# Let's see what is created

Output the binary in human readable form

Assembly listing

In C, C++, add the -S to the options

In Rust, add --emit=asm

These will create .s or assembly listing files

# What is linking?

Combines pieces of object code together

Resolves addresses

Computer must have known addresses  
to function

Let's look at link info using **nm** which  
lists the symbols found in binary objects.

```
nm <object_file_name>
```

# Can you look at program binary?

Sure

Use emacs in hexl-mode

It clearly show the file magic

This file is of type ELF

# ELF

The **Executable and Linkable Format (ELF)** is the standard binary file format for executables, object code, shared libraries, and core dumps in Linux and other Unix-like operating systems

# ELF Dual Purposes

Two views:

- **Linking View (Sections):** Used by compilers and linkers during program construction. It organizes the file into distinct **sections** containing code, data, and symbols
- **Execution View (Segments):** Used by the operating system loader at runtime. It groups sections into **segments** to define how memory pages should be allocated and protected
- [https://en.wikipedia.org/wiki/Executable\\_and\\_Linkable\\_Format](https://en.wikipedia.org/wiki/Executable_and_Linkable_Format)